PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Construcción y documentación de una Api con Express

Presentado a:	Yoli muñoz
Por Aprendiz:	**Beckan Hungria Rodriguez**
Ficha:	3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

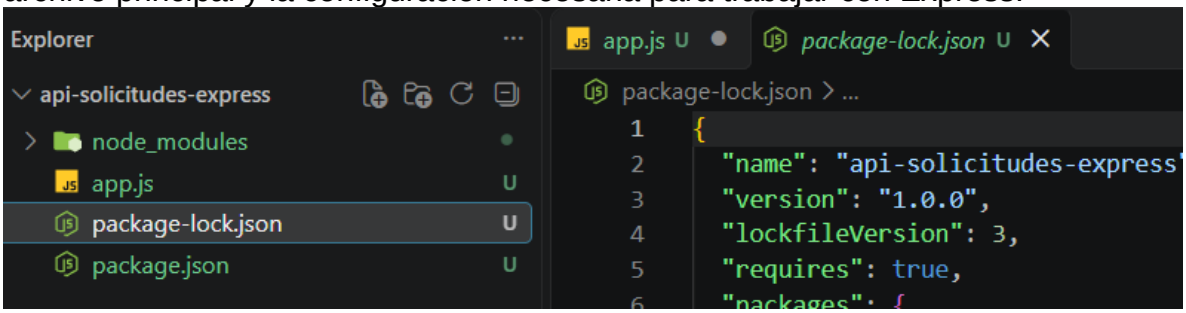
Popayán, día **20** de **octubre** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

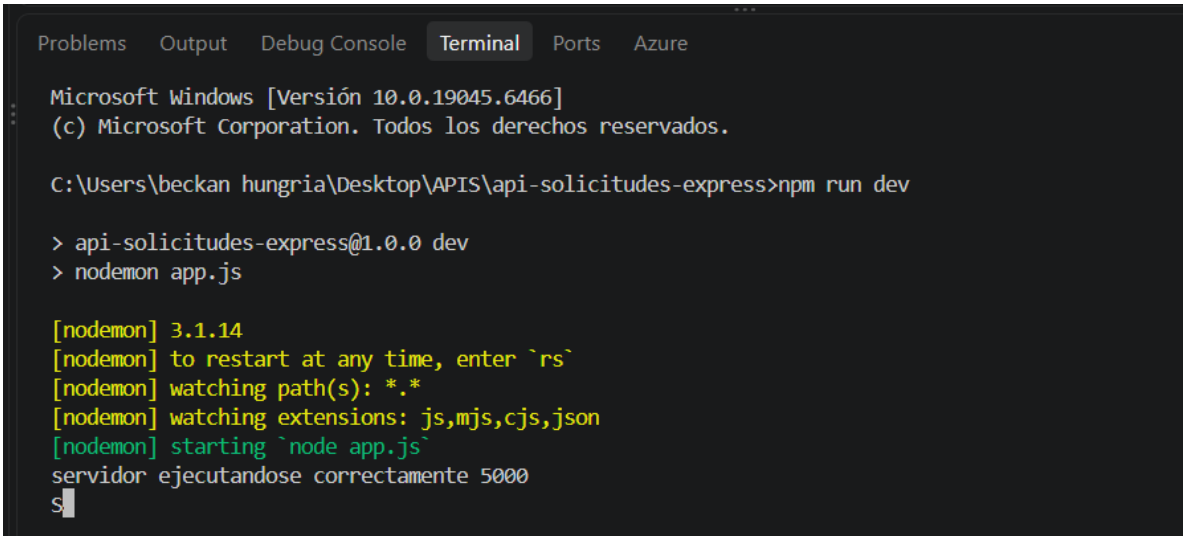
1. Actividad o ejercicio 1.

Pantallazo 1. Estructura del proyecto:

Aquí organicé los archivos del proyecto en Visual Studio Code, donde tengo el archivo principal y la configuración necesaria para trabajar con Express.



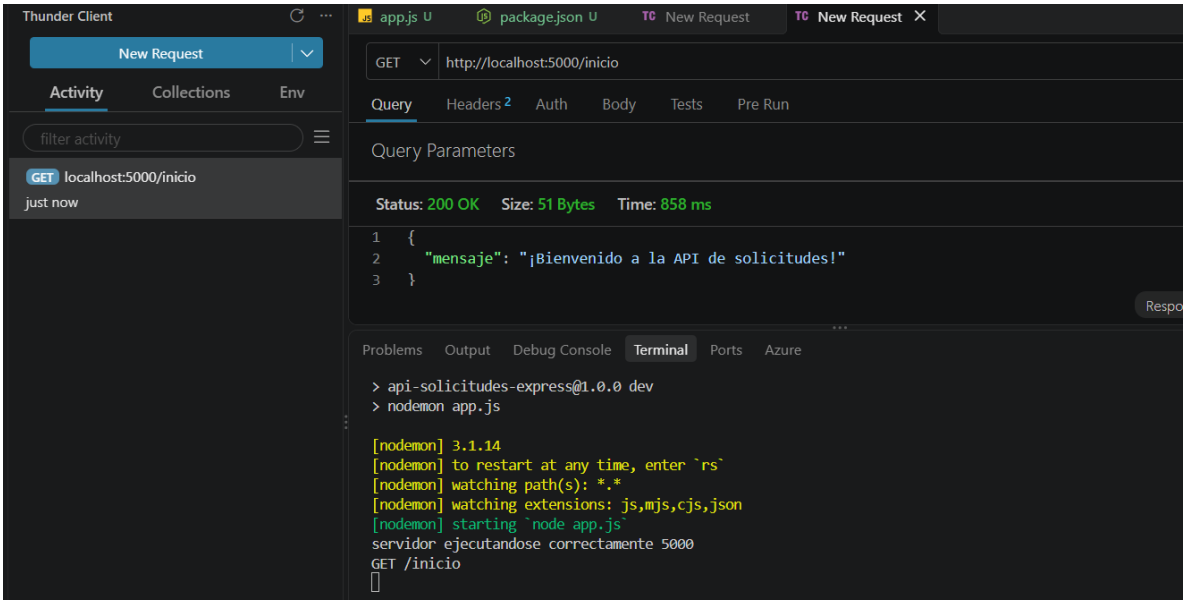
Pantallazo 2. Servidor funcionando Aquí se muestra que el servidor inició correctamente y quedó funcionando en el puerto 3000 para recibir las peticiones.



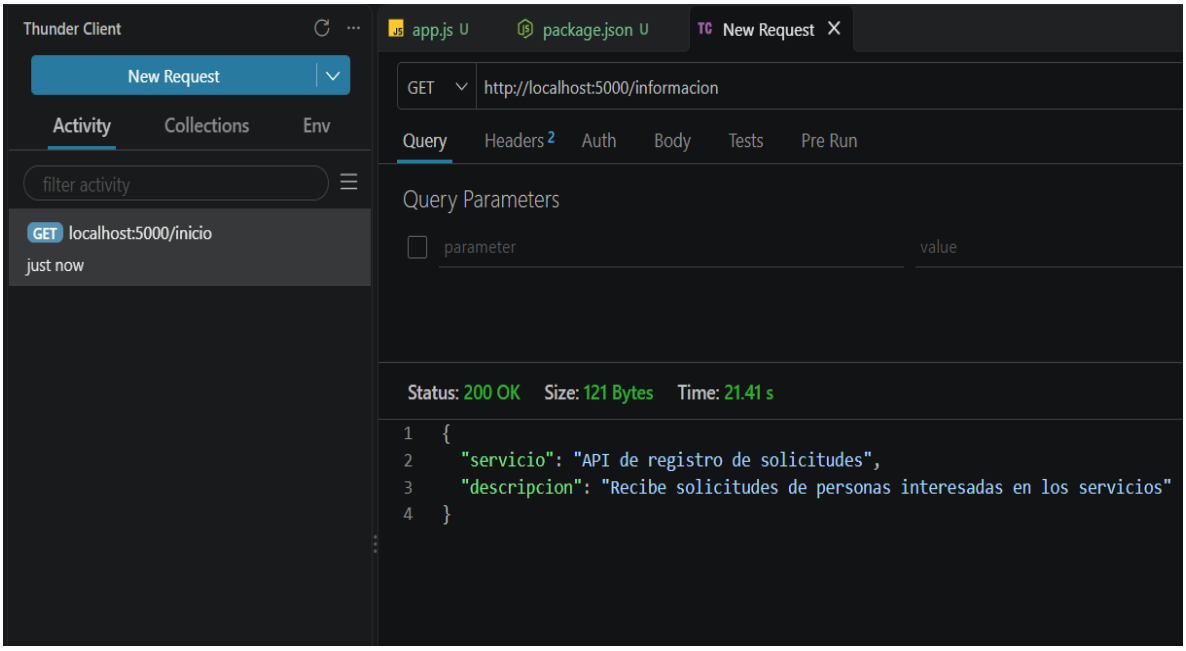
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Pantallazo 3. GET /inicio

En esta prueba comprobé que la ruta de inicio funciona y devuelve un mensaje de bienvenida en formato JSON con el código 200.

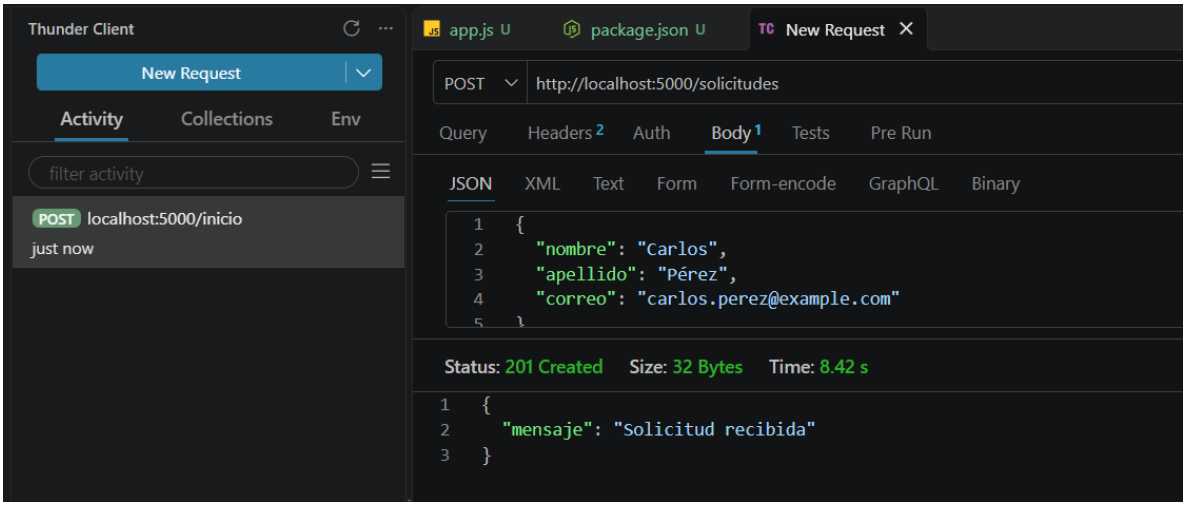


Pantallazo 4. GET /información Aquí verifiqué que la ruta devuelve correctamente la información general de la API y responde con el código 200.

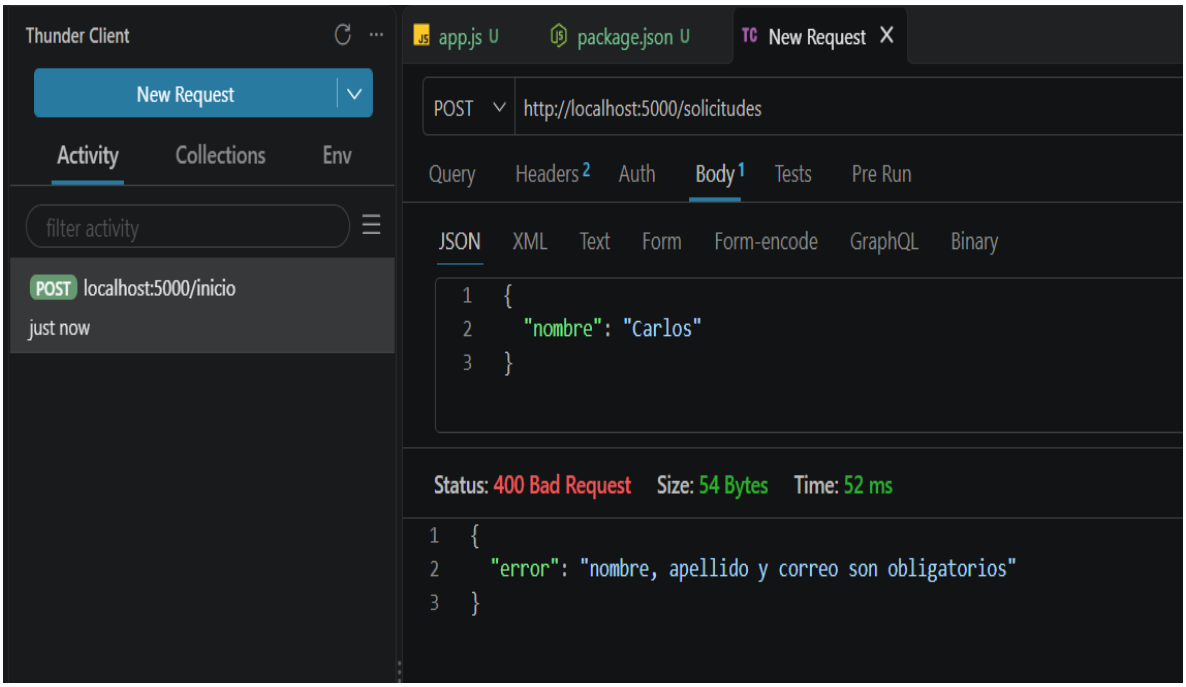


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Probar la ruta POST POST válido En esta prueba envié el nombre, el apellido y el correo correctamente. La API recibió la solicitud y respondió con el código 201.

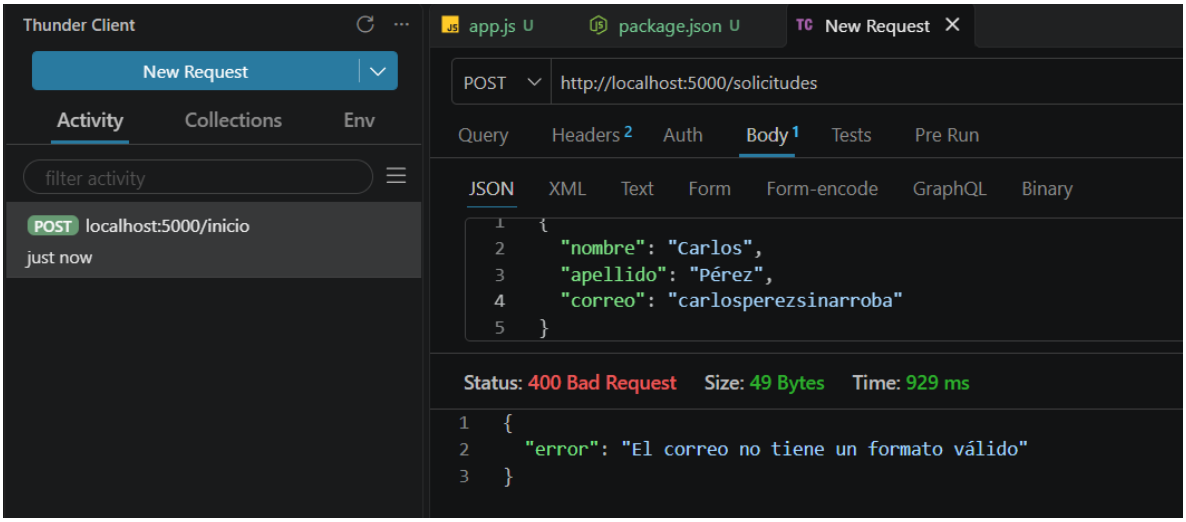


Pantallazo 6. Campos faltantes Aquí comprobé que la API identifica cuando falta un dato obligatorio y responde con el código 400 para indicar que la solicitud tiene errores.



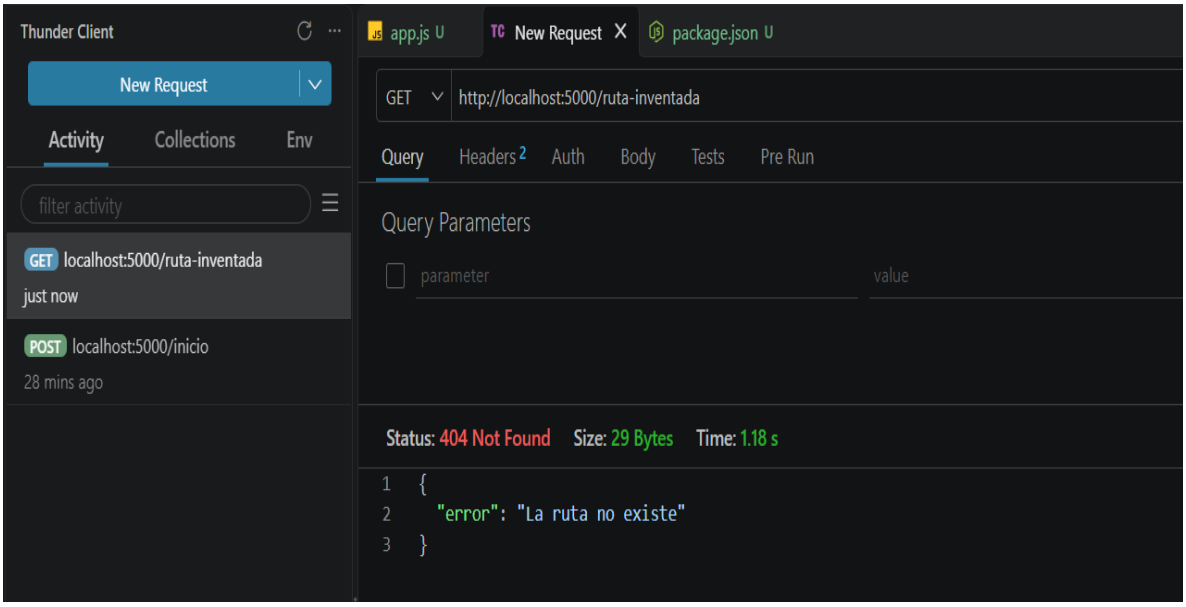
Pantallazo 7. Correo inválido

En esta prueba envié un correo con un formato incorrecto y comprobé que la API lo rechaza con el código 400.



Pantallazo 8. Ruta inexistente

Aquí consulté una ruta que no existe para comprobar que la API responde con el código 404 y muestra un mensaje de error.



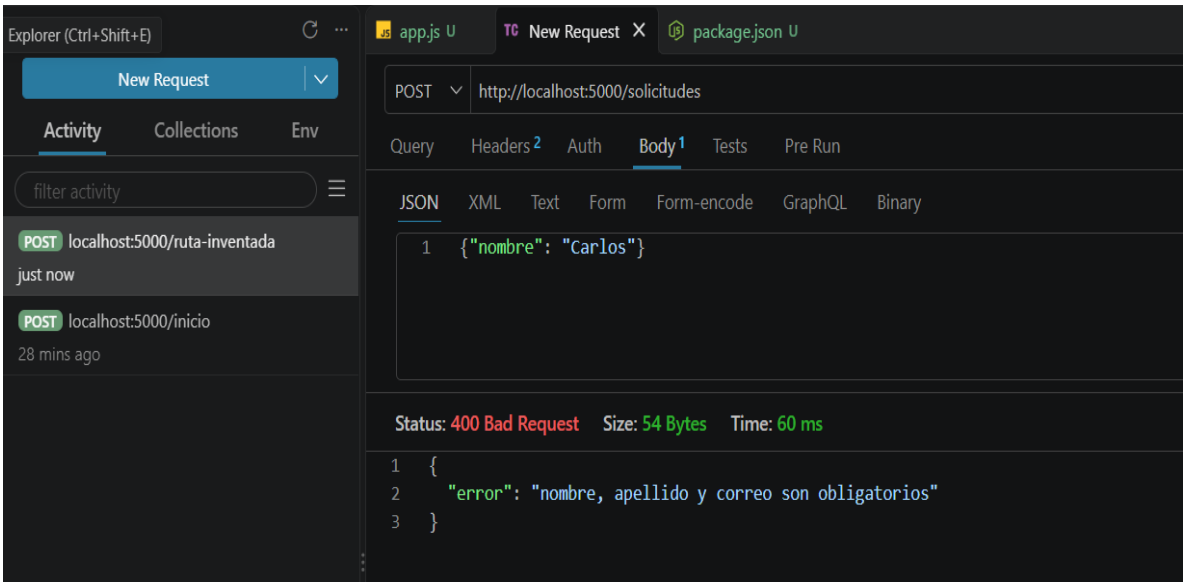
Pantallazo 9. Middleware propio

En esta captura se observa cómo el middleware registra en la consola el método HTTP y la ruta de cada petición que recibe el ser



Pantallazo 10. JSON mal formado

Aquí envié un cuerpo JSON incompleto para comprobar que la API detecta el error y responde con el código 400.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Investigación sobre middleware en Express

1. ¿Qué es un middleware en Express y en qué momento del ciclo de una petición se ejecuta?
Pues un middleware es una función que se mete en la mitad entre que llega la petición y que el servidor responde. Recibe req, res y next, y pienso que sirve para revisar o anotar cosas antes de que la petición llegue a la ruta. Se ejecuta apenas entra la petición, antes de la función de la ruta, y en el mismo orden en que uno los escribió en el código.

2. ¿Qué significan los objetos req y res?
req viene de request y es todo lo que el cliente me manda: el método (GET o POST), la ruta, los encabezados y el cuerpo con los datos. res viene de response y es lo que uso yo para contestarle, por ejemplo res.json() para mandar datos y res.status() para decir el código de estado.

3. ¿Cuál es la diferencia entre un middleware global, uno de una ruta específica y uno de manejo de errores?
El global se aplica a todas las peticiones sin importar la ruta, y se pone con app.use(). El de ruta específica solo se ejecuta cuando alguien entra a esa ruta en particular. El de errores es diferente porque tiene cuatro parámetros (err, req, res, next) y solo se activa cuando algo falla, así uno puede responder con un mensaje controlado y el servidor no se cae.

4. ¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía JSON?
Lo que hace es leer el JSON que manda el cliente en el cuerpo de la petición y volverlo un objeto de JavaScript dentro de req.body. Si no se configura, req.body llega sin definir y la API cree que no mandaron nada, aunque el cliente sí haya enviado los datos. Esto lo probé quitando esa línea y el POST empezó a dar error.

5. ¿Por qué importa el orden en que se registran los middleware y las rutas?
Porque Express los va ejecutando de arriba hacia abajo y el primero que responde termina el recorrido. Por eso express.json() tiene que ir antes de las rutas que leen req.body, el middleware que registra las peticiones va de primero para que las vea todas, y el 404 va de último, porque si lo pongo arriba tapa todas las rutas y todo daría que no existe.

6. ¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo?
Que la petición se queda colgada. El cliente espera y espera sin recibir nada hasta que se vence el tiempo. Esto pasa cuando uno se olvida de poner next() y tampoco responde con res.

7. ¿Cuál es la diferencia entre un middleware y la función que atiende una ruta?
El middleware normalmente prepara, revisa o registra algo y después pasa el control con next(). La función de la ruta es la que da la respuesta final al cliente. Pienso que el middleware es parte del camino y la ruta es el destino.

8. ¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite?
Entrarían datos incompletos o mal escritos a la API y eso puede causar errores o resultados raros. Además la API respondería que todo salió bien aunque no fuera así y el cliente nunca se daría cuenta. Y si falta una comprobación de acceso, cualquier persona podría hacer cosas que no le corresponden, entonces eso es un problema de seguridad.

9. Ejemplo cotidiano, sin código
Es como el portero de un edificio. Cuando llega alguien, el portero anota quién es y a qué hora llegó, y mira si puede pasar. Si todo está bien lo deja seguir hacia la oficina, y si no, no lo deja entrar y le explica por qué. El portero sería el middleware y la oficina a la que llega la persona sería la función de la ruta.

5. Middleware propio

¿Dónde lo ubiqué y por qué?
Lo puse al principio, antes de express.json() y antes de las rutas. Lo hice así porque quería que se ejecutara con todas las peticiones que llegan, incluso las que terminan en 404 o en error.

¿Qué cambia si el middleware se coloca después de las rutas?
Pues casi no registraría nada. Cuando una petición coincide con una ruta, esa ruta responde y ahí se acaba el recorrido, entonces nunca llegaría al middleware que quedó abajo. Solo registraría las peticiones que no coincidieron con ninguna ruta.

¿Cómo se transmite el control al siguiente middleware o a la ruta?
Con next(). Se llama al final del middleware y eso le dice a Express que ya terminé y que siga con lo que viene, ya sea otro middleware o la ruta. Si no lo pongo, la petición se queda trabada ahí.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Códigos de estado HTTP

200 OK
Lo uso en las rutas GET (/inicio y /informacion) cuando la consulta salió bien. Lo hice porque es el código que indica que la petición se procesó correctamente. Si falla algo, no devuelvo 200.

201 Created
Lo uso en el POST de /solicitudes cuando los tres datos vienen completos y el correo es válido. Pienso que corresponde porque con ese POST se está creando una solicitud nueva, y 201 es el código para cuando algo se crea.

400 Bad Request
Lo uso cuando el cliente manda datos faltantes o con formato incorrecto, por ejemplo si falta el correo o el correo no tiene arroba. Lo hice así porque el problema viene del cliente y no del servidor, y no tendría sentido responder con éxito.

404 Not Found
Lo uso cuando alguien consulta una ruta que no existe. Por eso este manejador va después de todas las rutas: si la petición no coincidió con ninguna, entonces llega acá.

500 Internal Server Error:
Lo uso cuando pasa un error inesperado del lado del servidor, o sea algo que no es culpa del cliente. En mi código lo maneja el middleware de errores que va de último. Investigué que se usa cuando el servidor falla de una forma que no se había previsto.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Cuadro Resumen de Rutas y Pruebas de la API:

Método	Ruta / URL	Propósito de la Prueba	Código HTTP	Resultado
GET	/inicio	Mensaje de bienvenida de la API	200 OK	Exitoso
GET	/informacion	Consulta de datos generales del servicio	200 OK	Exitoso
POST	/solicitudes	Registro con todos los datos correctos	201 Created	Exitoso
POST	/solicitudes	Validación de envío sin campos obligatorios	400 Bad Request	Exitoso
POST	/solicitudes	Validación de correo con formato inválido	400 Bad Request	Exitoso
GET	/ruta-inexistente	Manejo de error al consultar una ruta que no existe	404 Not Found	Exitoso
POST	/solicitudes	Envío de cuerpo con formato JSON mal escrito	400 Bad Request	Exitoso

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Cuadro de Autoevaluación o Conclusiones

Criterio de Evaluación	Cumple (Sí / No)	Observaciones / Evidencia
¿Se configuró correctamente el entorno de Node.js y Express?	**Sí**	Se utilizó Nodemon para el entorno de desarrollo y se estructuró el proyecto en el puerto 5000.
¿Se implementaron los endpoints GET y POST exigidos?	**Sí**	Se crearon las rutas de inicio, información y el registro de solicitudes con validaciones.
¿Se manejaron los códigos de estado HTTP de manera adecuada?	**Sí**	Se implementaron códigos 200, 201, 400, 404 y 500 según el tipo de respuesta o error.
¿Se documentó el proceso mediante capturas de pantalla?	**Sí**	Se recolectaron las evidencias visuales de cada una de las pruebas realizadas en Thunder Client y la consola.

Fuentes consultadas

- **Documentación oficial de Express, sección Using middleware: https://expressjs.com/en/guide/using-middleware.html**
- **Documentación oficial de Express, sección Error handling: https://expressjs.com/en/guide/error-handling.html**
- **MDN Web Docs, códigos de estado HTTP: https://developer.mozilla.org/es/docs/Web/HTTP/Status**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS